TCU4G · SERVER

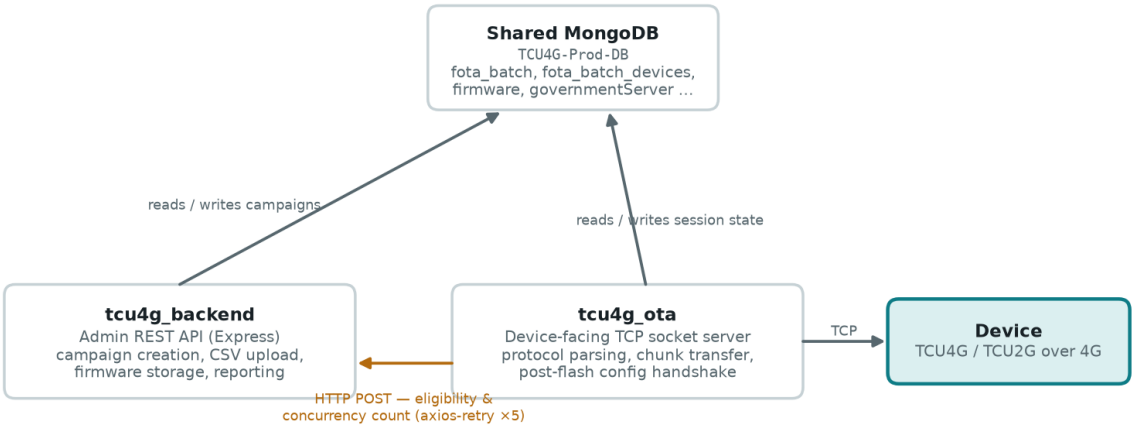
FOTA Workflow

How a firmware update travels from an admin's upload in tcu4g_backend to a confirmed flash on a device, over the raw TCP link owned by tcu4g_ota.

Raw TCP socket not MQTT — comma-delimited ASCII + 55AA binary frames	**1024 B chunks** + CRC-16/XMODEM on TCU4G	**base64 in MongoDB** no S3, no filesystem, no GridFS	**≤1000 concurrent** global flash concurrency cap	**3 retries** then the device is permanently aborted

01 How the two services connect

tcu4g_backend (admin API) and tcu4g_ota (device-facing socket server) don't call each other for the core workflow — they read and write the same MongoDB database. The only network call between them is a small HTTP check used to cap how many devices can flash at once.



tcu4g_ota owns the device connection; both services meet only at the shared database, plus one narrow HTTP check for flash concurrency.

02 Campaign setup

An operator arms a set of devices before any of them see a byte of firmware. Nothing reaches a device until step 3 explicitly starts the batch.

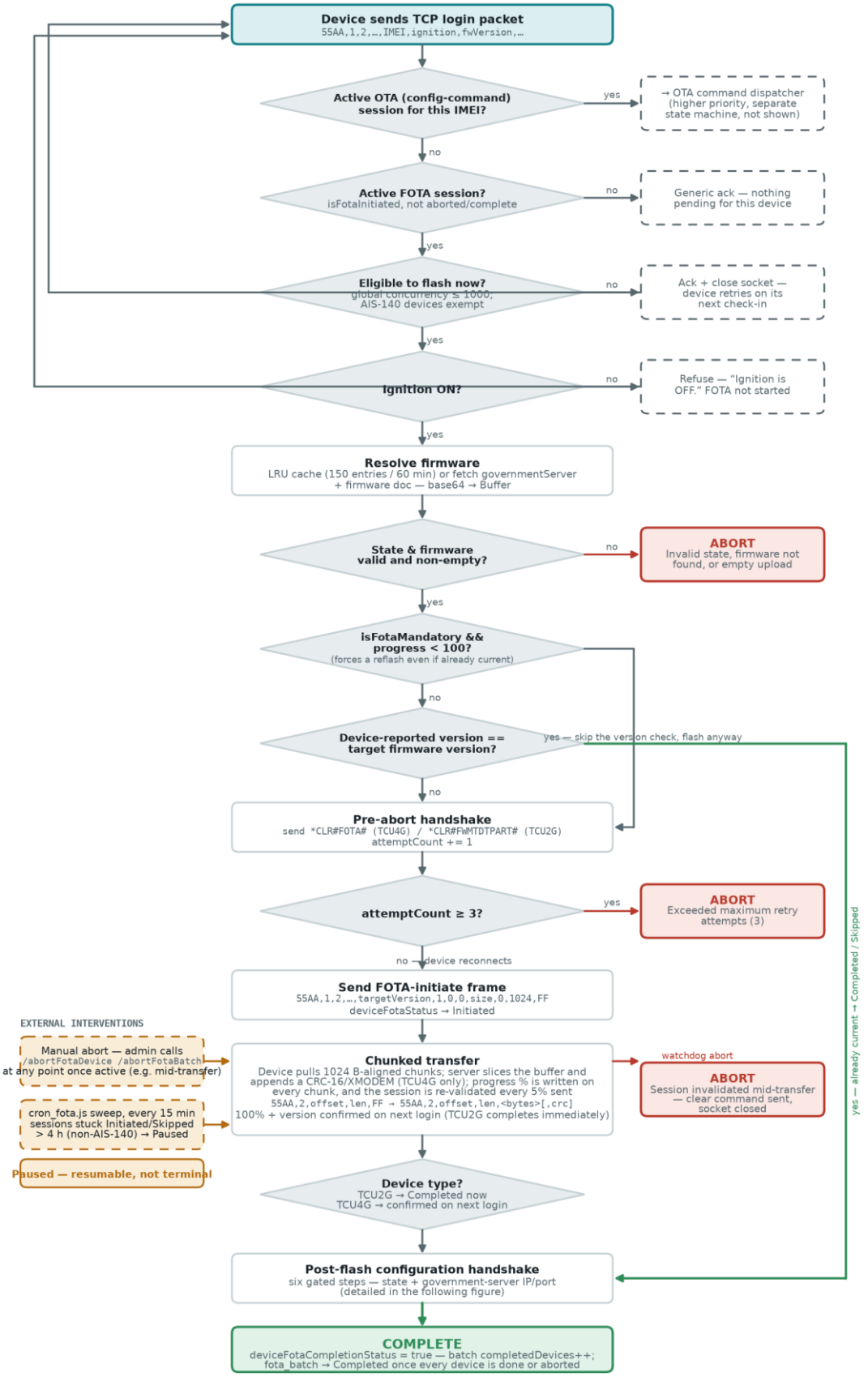
Step	What happens	Key fields
1 · Upload firmware	POST /api/govtServer/saveFirmwareData stores the binary as a base64 string on a firmware document, scoped to one state's governmentServer record.	`firmware.file (base64)` `firmware.firmwareVersion`
2 · Create batch	CSV or manual entry creates one	`fota_batch.status: Not-Started`

	fota_batch plus a fota_batch_devices row per device. Invalid UIN/IMEI/state/firmware rows are aborted immediately.	`deviceFotaStatus: Pending`
3 · Start FOTA	GET /startFota is the explicit “go” — it arms every device in the batch. An AIS-140 auto-created batch skips this and starts itself.	`isFotaInitiated: true` `fota_batch.status: In-Progress`

03 Device session — the state machine

Everything below runs inside tcpClientConnectionHandler.js each time an armed device dials in. A device that fails a soft check simply reconnects later and re-enters at the top; only the retry cap and firmware-resolution failures end a session permanently.

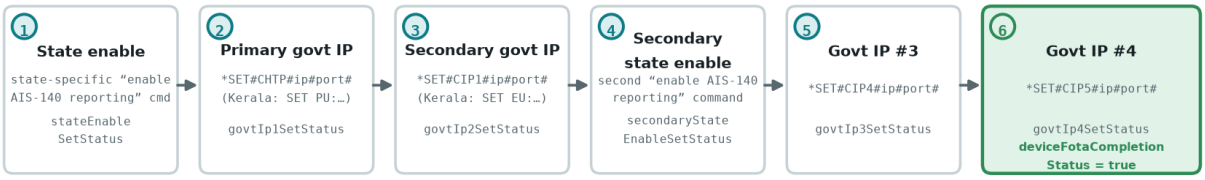
■ process step ■ decision ■ no-op / retry later ■ terminates as Aborted ■ terminates as Completed ■ external intervention


Device sends TCP login packet
55AA,1,2,…,IMEI,ignition,fwVersion,…
Active OTA (config-command) session for this IMEI?
yes
→ OTA command dispatcher (higher priority, separate state machine, not shown)
no
Active FOTA session?
isFotaInitiated, not aborted/complete
no
Generic ack — nothing pending for this device
yes
Eligible to flash now?
global concurrency ≤ 1000; AIS-140 devices exempt
no
Ack + close socket — device retries on its next check-in
yes
Ignition ON?
no
Refuse — "Ignition is OFF." FOTA not started
yes
Resolve firmware
LRU cache (150 entries / 60 min) or fetch governmentServer + firmware doc — base64 → Buffer
State & firmware valid and non-empty?
no
ABORT
Invalid state, firmware not found, or empty upload
yes
isFotaMandatory && progress < 100?
(forces a reflash even if already current)
no
Device-reported version == target firmware version?
yes — skip the version check, flash anyway
no
Pre-abort handshake
send *CLR#FOTA# (TCU4G) / *CLR#FWMTDTPART# (TCU2G)
attemptCount += 1
attemptCount ≥ 3?
yes
ABORT
Exceeded maximum retry attempts (3)
no — device reconnects
Send FOTA-initiate frame
55AA,1,2,…,targetVersion,1,0,0,size,0,1024,FF
deviceFotaStatus → Initiated
EXTERNAL INTERVENTIONS
Manual abort — admin calls /abortFotaDevice /abortFotaBatch at any point once active (e.g. mid-transfer)
cron_fota.js sweep, every 15 min sessions stuck Initiated/Skipped > 4 h (non-AIS-140) → Paused
Paused — resumable, not terminal
Chunked transfer
Device pulls 1024 B-aligned chunks; server slices the buffer and appends a CRC-16/XMODEM (TCU4G only); progress % is written on every chunk, and the session is re-validated every 5% sent
55AA,2,offset,len,FF → 55AA,2,offset,len,<bytes>[,crc]
100% + version confirmed on next login (TCU2G completes immediately)
watchdog abort
ABORT
Session invalidated mid-transfer — clear command sent, socket closed
Device type?
TCU2G → Completed now
TCU4G → confirmed on next login
Post-flash configuration handshake
six gated steps — state + government-server IP/port (detailed in the following figure)
COMPLETE
deviceFotaCompletionStatus = true — batch completedDevices++; fota_batch → Completed once every device is done or aborted
yes — already current → Completed / Skipped

The device-session state machine inside tcu4g_ota's TCP handler. A mandatory batch (isFotaMandatory) forces a real reflash even if the device already reports the target version, until progress reaches 100%. Amber boxes are outside interventions that can pause or abort a session already in flight.

04 Post-flash configuration handshake

Whether a device just finished flashing or was already on the right version, it still has to pick up its AIS-140 government-server settings before deviceFotaCompletionStatus can flip to true. Each step is gated on the previous one's status being Set, Skipped, or NA, and is itself confirmed on the device's next check-in before the following command goes out.



Six AIS-140 configuration commands, sent and confirmed one round-trip at a time; the last one closes out the device's FOTA session.

05 Reference

Status enums

Entity	Field	Values
fota_batch	status	Not-Started In-Progress Completed Aborted
fota_batch_devices	deviceFotaStatus	Pending Initiated Completed Skipped Paused
fota_batch_devices	…SetStatus (×6)	Pending Set Skipped Triggered NA
ota_batches	status	In-Progress Completed Aborted
ota_batch_device_commands	status	Pending Triggered Completed Mark As Completed Aborted

Key constants

Chunk size	1024 B
Chunk checksum	CRC-16/XMODEM (TCU4G)
Handshake retry cap	3 attempts
Firmware cache	150 entries / 60 min TTL
Flash concurrency cap	1000 devices
Socket idle timeout	120,000 ms
Stale-session pause window	4 h, swept every 15 min

OTA command watchdog	`60,000 ms + 5 min sweep`

06 What's not there

Worth stating plainly, since these are easy to assume by default:

- **No MQTT —** Both services were grepped end to end — device transport is a raw TCP socket with a proprietary ASCII/CSV protocol, not a pub/sub broker.
- **No object storage —** Firmware binaries live as a base64 string on a MongoDB document, not S3, disk, or GridFS.
- **No signed-URL download —** The binary is pushed inline over the same TCP session, chunk by chunk, pulled by the device — not a link to fetch over HTTP.
- **No automatic rollback —** A device that exceeds the retry cap is permanently aborted from that batch; recovering it means an admin creates a brand-new batch.
- **OTA takes priority on the wire —** The parallel “OTA” system pushes config commands (APN, custom AT strings) over the same connection and is checked before FOTA on every login.